

System Analysis and Design

Eighth Edition

Alan Dennis, Barbara Wixom, Roberta M. Roth



Chapter 4

Understanding Processes with Use Cases and
Process Models

Objectives

- Explain the purpose of use cases in the analysis phase of the SDLC.
- Describe the various parts of a use case and the purpose of each part.
- Describe how use cases contribute to the functional requirements.
- Describe how use cases inform the development of test plans.
- Explain the process used to create a use case.
- Explain the rules and style guidelines for data flow diagrams.
- Describe the process used to create data flow diagrams.
- Create data flow diagrams.

Introduction

- A key aspect of determining the requirements for the new system is understanding the ***user requirements***
 - The things the users need to accomplish with the new system
- Use cases help us understand and clarify the users' required interactions with the system and can help us more fully understand the functional requirements of the new system
- A use case represents how a system interacts with its environment
- Use cases are especially valuable for business system applications and websites
- Process models have been a part of structured systems analysis and design techniques for many years

What Is a Use Case?

- A use case depicts a set of activities performed to produce some output result
- Each use case describes how an ***event triggers*** actions performed by the system and the user
 - Everything in the system can be thought of as a response to some trigger event

Basic Information

- Each use case has a *name* and *number*, and brief *description*.
- The *priority* may be assigned to indicate the relative significance.
- The *actor* refers to a person, another system, or a hardware device that interacts with the system to achieve a useful goal.
- The *trigger* for the use case – the event that causes the use case to begin.
- Events triggers can be *external* or *temporal*

Create Preliminary Custom Drone Order Use Case—Casual Format

Use Case Name: Create preliminary custom drone order	ID: UC-6	Priority: High
Actor: Customer		
Description: The customer selects and customizes a commercial drone to purchase		
Trigger: Customer wants to purchase a commercial drone		
Type: ☒ External ☐ Temporal		

Preconditions

- It is important to define clearly what needs to be accomplished before each use case begins
- Preconditions define the state the system must be in before the use case commences

Preconditions:

1. The customer is authenticated by logging in to his account
 2. The Sales System Order Processing application is online
-

Normal Courses

- The normal course lists the steps that are performed when everything flows smoothly in the system
- Sometimes called the *happy path*

Normal Course:

- 1.0 Order a customized drone
 1. The customer selects a base model drone from a list of models
 2. The system provides availability status for that model (in stock, out of stock)
 3. For out of stock status, system displays expected date available
 - a. Customer accepts future availability date; proceed to step 4
 - b. Customer rejects future availability date; return to step 1
 4. The system displays a list of options and upgrades for the selected model
 5. The customer selects desired model options and upgrades
 6. Preliminary order with cost estimate is created and displayed
 7. Customer may return to step 4, confirm order, save for future consideration, or exit without saving
 8. Unconfirmed orders are stored in Unconfirmed Custom Order datastore
 9. Confirmed orders are saved in Confirmed Custom Order datastore
 10. Shop manager is notified of Confirmed Order requiring approval
-

Postconditions

- In this section of the use case, we define the final products of this use case
- These postconditions also serve to define the preconditions for the next use case in the series

Postconditions:

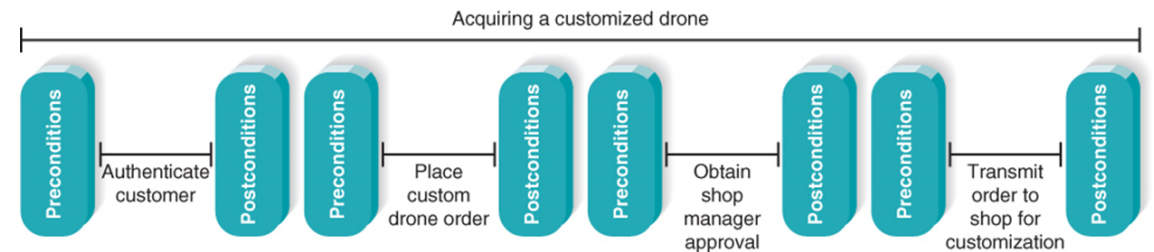
1. Unconfirmed order is stored in Unconfirmed Custom Order datastore
 2. Confirmed order is stored in Confirmed Custom Order datastore
 3. Shop manager sent notice of Confirmed Order requiring approval
-

Exceptions

- To be complete, a use case should describe any error conditions or exceptions that may occur as the use case steps are performed
- These are not normal branches in decision logic but are unusual occurrences or errors that could potentially be encountered and will lead to an unsuccessful result
- We want to be sure that the system does not fail while in use because of an error

Use Cases in Sequence

- Uses cases often performed in sequence
- No single use case should be too large
- Important to define initial and ending states



Additional Use Case Issues

- Alternative paths
- Summary of inputs and outputs
- Frequency of use
- Business rules
- Special requirements
- Assumptions
- Notes and issues

More Elaborate Use Cases are Especially Valuable When...

- User representatives are not actively engaged with the development team throughout the project
- The application is complex and has a high risk associated with system failures
- Comprehensive test cases will be based on the user requirements
- Collaborating remote teams need a detailed, shared understanding of the user requirements

Use Case Practical Tips

- Use gradual refinement
- Concentrate on describing the user's objectives with the system completely and accurately
- Keep both audiences in mind – users and developers
- Create use cases only when needed to clarify what the system must do from the user's perspective
 - Not needed for simple events

Use Cases and the Functional Requirements

- Use cases are useful tools to clarify user requirements
- Use cases convey only the user's point of view
- Transforming the user's view into the developer's view through functional requirements is one of the system analyst's key contributions
- The derived functional requirements tell the developers more about what the system must do

Steps for Writing for Use Cases

1. Identify the use cases
2. Identify the major steps within each use case
3. Identify elements within steps
4. Confirm the use case

Creating Use Cases

- Identify events the system must respond to – develop event-response list
- Create use case form for the complex events
- For each use case:
 - Identify the major steps
 - Identify elements with each major step (inputs and outputs)
 - Confirm use case with users through role-playing
- Revise functional requirements as needed

Identify the Major Steps for Each Use Case

- The analyst should ask the users what tasks need to be completed before the use case steps can begin
- Next, the user–system interactions should be outlined as a series of steps in the Normal Course section of the form
- Each step should be about the same size as the others
- Occasionally, a use case is so simple that further refinement is not needed
- Once the steps have been outlined at the proper level of detail, the postconditions can be completed

Identify Elements within Steps

- The last column (“Information for Steps”) must be completed, and arrows may be drawn to describe inputs and outputs from the steps
- The goal at this point is to identify the major inputs and outputs for each step
- The users and analysts then return to the steps in the use case and begin tracing the flow of the steps
- It is not unusual at this point for users to discover that they forgot to list the entire steps
- The Summary area for inputs and outputs found at the end of the use case form is completed once the team is satisfied with the steps, inflows, and outflows

Confirm the Use Case

- The final step is for the users to confirm that the use case is correct as written
- Review the use case with the users to make sure that each step and each input and output are correct